

CloudPilot AI

Multi-Agent AWS Security & Operations Assistant

Technical Documentation

Ritvik Indupuri

Abstract

CloudPilot AI combines an AWS operations interface with a Deno agent orchestrator, Anthropic model requests, intent-specific skills and application persistence. Its central module defines tool contracts, safety review, domain helper routines and streamed response handling. Hosted configuration uses Supabase; the local workflow supplies a Deno API and SQLite database.

The implemented agent loop dispatches approved batches to aws-agent-tools. That endpoint and the separate scanner, operations, executor and credential-exchange entry files are empty in the public revision reviewed here. The documentation therefore describes the implemented orchestration and supporting code without claiming a complete deployed AWS execution chain.

Contents

Abstract	2
1 System overview	3
2 System architecture	4
3 Agent request lifecycle	5
4 Skills and tool contracts	6
5 Model routing and safety	6
6 Data and evidence	7
7 Deployment and configuration	7
8 Implementation cross-reference	8
9 Next steps	8

1 System overview

The browser provides authentication, conversations, security findings, costs, drift, runbooks, organization operations, approvals, compliance views, settings and a Skills Catalog. The agent function validates messages and supplied AWS credential fields, resolves a skill and coordinates model requests. Source helper routines cover AWS inspection and mutation domains, while the active model loop uses a separate HTTP tool-dispatch contract.

1.1 Runtime components

Component	Implemented responsibility
React / Vite	Application pages, request composition, incremental responses and reports.
aws-agent	Input checks, skill selection, Anthropic requests, safety audit, tool batching and SSE.
Local Deno server	Compatible local authentication/data routes backed by cloudpilot.db.
Supabase migrations	Hosted PostgreSQL schema, ownership policies and operational records.
deploy/	CloudFormation, Terraform and Lambda source for an event-driven AWS mini agent.

1.2 Execution boundary

A declared tool is an interface contract. A helper function is source implementation. Neither establishes that an independently deployed endpoint accepts and executes the call. This distinction matters here because the central loop posts to an empty tool-service entry point rather than invoking its inline helper routines directly.

2 System architecture

The diagram connects the browser to runtime routing and the central agent, then shows the model and tool-service boundaries. The supporting AWS helpers and in-VPC deployment are separate source components. They are not shown as a verified continuation of the tool endpoint. PostgreSQL and SQLite represent alternative persistence modes.

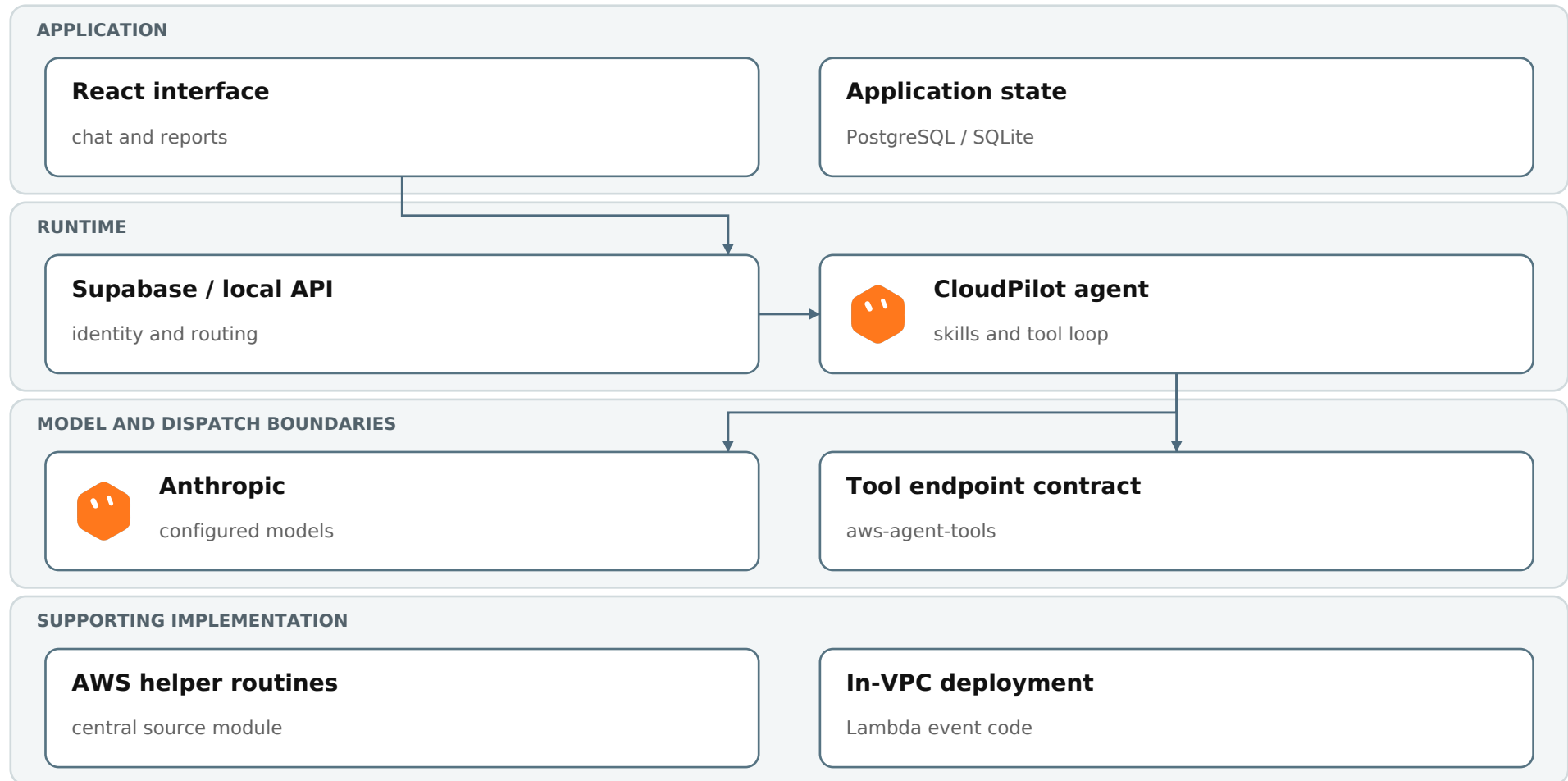


Figure 1: Application, agent and integration boundaries.

3 Agent request lifecycle

The request carries AWS credential fields into validation; credential exchange is not an implemented prerequisite inside this loop. Skill resolution and Fast/Deep selection prepare the model context. Proposed calls enter runSafetyAudit before batch dispatch. Rejections are appended as tool errors for another iteration. A successful downstream response supplies tool messages for continued reasoning; the endpoint implementation must be supplied for that path to complete.

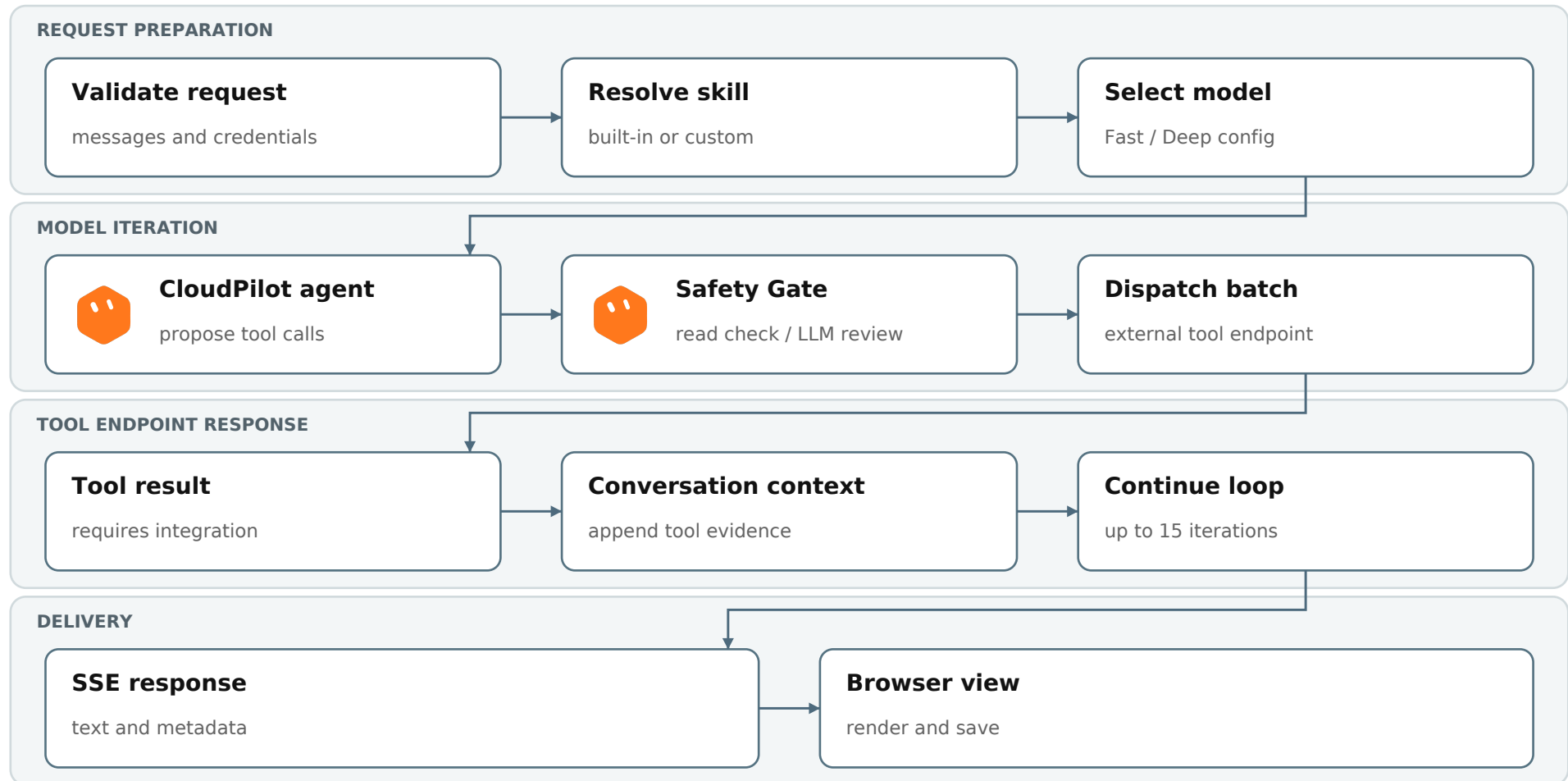


Figure 2: Orchestration and response handling in the central agent.

4 Skills and tool contracts

The source defines nine built-in intent profiles: security audit, cost analysis, drift detection, Organizations, incident response, attack simulation, event automation, resource queries and general assistance. Profiles supply system instructions and intent-scoped tools. Custom skills can be explicitly selected or matched by configured keywords; their persona supplement and allowed tool names modify the request context.

Tool contract	Domain
execute_aws_api	Generic AWS request
manage_iam_access / manage_security_group_rule	IAM and network changes
run_unified_audit	Security assessment
run_attack_simulation / run_evasion_test	Security exercises
manage_cost_rule / run_cost_anomaly_scan	Cost rules and anomaly analysis
manage_drift_baseline / run_drift_detection	Configuration comparison
run_org_query / manage_org_operation	Organization inspection and changes
manage_runbook_execution	Runbook state
manage_event_response_policy / replay_cloudtrail_events	Event policy and replay

These 15 definitions describe the arguments exposed to the model. The central file also contains domain algorithms and SDK helpers. The separate tool-service contract must connect those capabilities to the active loop before end-to-end execution can be established.

5 Model routing and safety

Fast mode reads ANTHROPIC_MODEL and Deep mode reads ANTHROPIC_DEEP_MODEL. The source default strings are claude-sonnet-5 and claude-opus-5. They are configuration identifiers, not evidence that those identifiers are available to a particular Anthropic account. The request helper contains fallback model identifiers and retries for selected failures.

runSafetyAudit first classifies the proposed batch using tool names and operation patterns. Batches recognized as read-only bypass the separate model review. Other batches are submitted to an Anthropic safety prompt. Non-success responses, parsing errors and non-boolean approval values result in rejection. This is a control inside the central loop, not a universal authorization boundary for every AWS entry point.

IAM permissions on the supplied AWS principal remain the provider-side authorization boundary. The source contains additional confirmation, risk classification and idempotency helpers, but their presence does not prove they execute on every dispatched request.

5.1 Response handling

The loop is bounded to 15 iterations. Metadata carries execution progress and safety outcomes over SSE; tool errors can be returned to the model for recovery. The frontend renders incremental output and associates conversation records with user-visible history.

6 Data and evidence

Hosted migrations define conversations and messages, audit and automation records, drift and cost state, runbook steps, approval records, organization membership and compliance evidence.

PostgreSQL row-level policies apply in the hosted deployment. The local gateway uses SQLite and its own session handling; it does not reproduce every PostgreSQL policy, trigger, realtime or scheduling feature.

The central source contains CloudWatch Logs and S3 Object Lock helper code as well as platform audit operations. Successful multi-destination logging and WORM retention depend on executed code paths, IAM permissions, destination configuration and successful writes. A source helper or attempted upload alone does not establish immutable evidence.

6.1 Local controls

The local server contains SQL identifier and operation allowlists, PBKDF2-SHA256 password hashing with 100,000 iterations, random session tokens and request/query bounds. The central agent declares an origin-specific CORS value. These are scoped source controls; no claim is made that all empty Edge Function entry files implement equivalent checks.

7 Deployment and configuration

The local container workflow supplies Node and Deno, runs the frontend and local API, and writes cloudpilot.db. Hosted operation uses the configured Supabase project and its deployed functions. The in-VPC deployment assets define a separate event-driven Lambda path with EventBridge, SNS, CloudWatch Logs and an SQS dead-letter queue.

```
cp .env.example .env
# Set the required runtime values.
docker compose up --build
```

Configuration	Purpose
VITE_SUPABASE_URL	Browser API endpoint.
VITE_SUPABASE_PUBLISHABLE_KEY	Browser project key.
SUPABASE_URL	Server-side Supabase endpoint.
SUPABASE_SERVICE_ROLE_KEY	Administrative server access.
SUPABASE_ANON_KEY	Downstream function request key.
ANTHROPIC_API_KEY	Model and safety-review requests.
ANTHROPIC_MODEL / ANTHROPIC_DEEP_MODEL	Configured Fast and Deep model identifiers.
ALLOWED_ORIGIN	Origin accepted by the central function.

Starting the container does not fill in the empty downstream services. Deployment verification must separately establish tool dispatch, credential handling, AWS calls and persistence under the intended runtime.

8 Implementation cross-reference

Source reference: CloudPilot-AI---AWS-Security-Agent, revision c4e4d0c3da18. The review covers source and configuration; it does not certify a live deployment or every runtime outcome.

Source	Responsibility
src/	Application routes and components.
src/pages/Skills.tsx	Skills Catalog and custom personas.
src/pages/InVpcAgent.tsx	In-VPC agent interface.
supabase/functions/aws-agent/index.ts	Active orchestration, contracts and inline domain helpers.
supabase/functions/aws-agent-tools/index.ts	Referenced dispatch entry; empty in reviewed revision.
aws-agent-scanner, aws-agent-ops, aws-executor	Separate function entries are empty in reviewed revision.
local-server.ts	Local API, authentication and SQLite.
supabase/migrations/	Hosted schema and policies.
deploy/	Event-driven AWS deployment assets.

9 Next steps

Implement and validate the downstream tool service, establish the credential-exchange path and test representative read and mutation operations against a controlled AWS account. Confirm model identifiers with the configured provider account, bind approvals to exact requests, and verify successful audit writes and retention. Add integration tests across both persistence modes before describing the full chain as operational.